

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 06 -

PADRÕES DE CÓDIGO E ESTILO

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	6
MERCADO, CASES E TENDÊNCIAS	18
O QUE VOCÊ VIU NESTA AULA?	19
REFERÊNCIAS.....	20

EMSE

O QUE VEM POR AÍ?

Você já se deparou com um código tão confuso que nem mesmo o autor consegue entendê-lo depois de algum tempo? Muitos(as) cientistas de dados acreditam que “programar é só um meio para chegar aos resultados”, mas esquecem que código é lido muito mais vezes do que é escrito. Padrões de código e estilo surgem exatamente para resolver esse problema, tornando o código mais legível, compartilhável e fácil de manter.

Nesta aula, vamos refletir sobre como convenções de estilo (como a PEP 8 no Python) e princípios de código limpo ajudam a evitar dores de cabeça. Veremos exemplos de problemas reais causados por código mal organizado e como boas práticas podem preveni-los, preparando você – cientista de dados – para escrever códigos com qualidade digna de um(a) engenheiro(a) de software.

Ao longo do material, discutiremos soluções para tornar seu código limpo e profissional. Você entenderá como nomes de variáveis bem escolhidos e formatação consistente facilitam a vida de todo mundo no projeto. Exploraremos também princípios consagrados como DRY (Don't Repeat Yourself – não se repita), KISS (Keep It Simple, Stupid – mantenha a simplicidade) e Single Responsibility (cada módulo com uma única responsabilidade) – mostrando casos de uso em que eles salvam o dia. Por fim, mergulharemos nos padrões de projeto mais usados (Strategy, Observer, Factory etc.), entendendo quando aplicá-los para resolver problemas comuns de forma elegante sem cair no over-engineering.

Prepare-se: esta aula vai conectar os pontos entre programação limpa e ciência de dados de alto nível, com muitos insights e boas práticas para pensar! (Ah, e não se esqueça de assistir às videoaulas correspondentes – elas trazem demonstrações práticas que complementam este conteúdo!)

HANDS ON

Nos vídeos desta aula, você verá na prática como aplicar esses conceitos de estilo e organização de código. A proposta é mostrar, passo a passo, como pequenas mudanças podem tornar um código bagunçado em um código limpo e eficiente. Por exemplo, vamos pegar uma tarefa comum em ciência de dados – comparar modelos de Machine Learning – e demonstrar como refatorar funções duplicadas em uma única função genérica, seguindo o princípio DRY de não repetir código.

Utilizaremos a linguagem Python (uma das mais usadas em ciência de dados) para implementar a solução, aproveitando sua simplicidade e clareza. A videoaula mostrará todo o processo: primeiro identificamos um “code smell” (cheiro de problema) – no caso, trechos de código muito parecidos se repetindo – e então aplicamos melhorias graduais, comentando cada etapa.

Não perca: assista às videoaulas e tente reproduzir os passos por conta própria, pausando quando necessário, pois colocar a mão no código é a melhor forma de aprender! Após, volte aqui e confira o código final comentado a seguir, que resume o que foi feito no hands on.

Para ilustrar, imagine que queremos avaliar dois algoritmos de classificação (por exemplo, AdaBoost e Random Forest) no famoso dataset do Titanic. Uma implementação apressada poderia criar duas funções praticamente idênticas – uma para cada modelo –, violando o DRY ao duplicar lógica. No hands on, refatoramos esse cenário criando uma única função genérica `avaliar_modelo`, que recebe o modelo como parâmetro.

Assim, tornamos o código mais conciso e fácil de expandir para outros algoritmos. A seguir temos a versão refatorada em Python, usada na demonstração em vídeo, seguindo as convenções da PEP 8 (note os nomes claros e indentação de 4 espaços):

```
def avaliar_modelo(dados, ClasseModelo):
    X = dados[["is_male", "SibSp", "Pclass", "Fare"]]
    y = dados["Survived"]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
    modelo = ClasseModelo()
    modelo.fit(X_train, y_train)
    return modelo.score(X_test, y_test)

# Exemplo de uso:
acuracia_rf = avaliar_modelo(df, RandomForestClassifier)
acuracia_ada = avaliar_modelo(df, AdaBoostClassifier)
```

Código-fonte 1 – Função em Python aplicando o princípio DRY para avaliar diferentes modelos de forma genérica

Fonte: Elaborado pelo autor (2025)

No trecho anterior, `avaliar_modelo` encapsula a lógica de treino e teste, evitando duplicação de código para cada algoritmo. Basta passar a classe do modelo desejado (`RandomForestClassifier` ou `AdaBoostClassifier`, previamente importadas do `scikit-learn`) que a função cuida do resto. Esse tipo de refatoração simples deixou o código mais flexível: se amanhã quisermos testar um terceiro algoritmo, podemos reutilizar a mesma função, mantendo nosso código sequinho (sem repetições).

Perceba também o estilo Python adotado – nomes em `snake_case`, indentação consistente – tudo conforme a PEP 8. Esses cuidados tornam o código mais legível para você e sua equipe, como foi demonstrado na videoaula. Lembre-se de revisar os vídeos e praticar esse exemplo; tente modificar a função para aceitar outros parâmetros (como métricas ou features diferentes) e observe como a clareza do código ajuda nessas evoluções.

SAIBA MAIS

Vamos agora nos aprofundar nos conceitos apresentados, construindo uma base teórica sólida sobre padrões de código e estilo. Nesta seção, vamos aumentar o nível de detalhe: entraremos em convenções de estilo (e por quê elas importam), destrincharemos princípios de código limpo com analogias e matemática simples e exploraremos padrões de projeto com olhos críticos – sempre conectando com situações de ciência de dados.

Prepare-se para uma abordagem técnica, porém descontraída, repleta de exemplos e até um pouco de história do desenvolvimento de software. Se você ainda não o fez, assista às videoaulas correspondentes antes de seguir, pois elas ajudarão a dar vida prática a muitos dos tópicos aqui discutidos. Vamos lá!

Convenções de estilo: escrevendo código para pessoas

Uma das primeiras coisas que um(a) cientista de dados nota ao entrar em projetos de software maiores é a existência de guides de estilo – documentos que ditam regras de formatação e nomenclatura. Em Python, o guia oficial é o PEP 8 – Style Guide for Python Code, criado em 2001 por Guido van Rossum e colaboradores. Por que tanta disciplina? Porque “código limpo é código que humanos entendem”, já dizia Martin Fowler. Ou seja, não basta funcionar; o código precisa ser compreensível e padronizado para outros desenvolvedores e desenvolvedoras (e seu eu do futuro) conseguirem dar manutenção facilmente.

A PEP 8 define convenções como: nomes de variáveis e funções em `snake_case` (letras minúsculas_com_underlines), nomes de classes em `CamelCase` iniciando em maiúscula, uso consistente de 4 espaços para indentação, limite de 79 caracteres por linha, entre outras recomendações. Vamos a um exemplo simples: em vez de nomear uma variável como `X` ou `df_final2` (nomes crípticos que nada dizem), preferimos algo como `dados_normalizados` ou `resultado_modelo`. Assim, o nome já conta uma história do que é aquele objeto, dispensando comentários redundantes. Do mesmo modo, funções devem ter nomes verbosos que indiquem ação, por exemplo `calcula_variancia()` ao invés de `cv()`.

Esse investimento em nomenclatura clara paga dividendos: estudos mostram que seguir a PEP 8 melhora a colaboração e até a empregabilidade, pois demonstra profissionalismo. Afinal, em projetos de ciência de dados colaborativos, outros podem nunca ter te conhecido pessoalmente, mas vão precisar ler e usar seu código. Ter um estilo consistente (mesmo que aparentemente trivial, como uso de espaço em volta de operadores ou quebras de linha estratégicas) reduz a “carga cognitiva” de quem lê, permitindo que o foco esteja na lógica e não na formatação.

Outra vantagem das convenções é possibilitar ferramentas de verificação automática. Linters como flake8 ou formataadores automáticos como black examinam seu código e apontam violações ao estilo padrão. Em projetos empresariais, é comum integrar essas ferramentas no pipeline de integração contínua: se o código não estiver no padrão, ele nem vai para produção. Em ciência de dados, adotar essas práticas de engenharia traz robustez ao trabalho de pesquisa.

Pense em notebooks Jupyter desorganizados que você talvez já tenha visto (ou escrito): células fora de ordem, variáveis temporárias chamadas tmp ou data2 espalhadas... Seguir um padrão desde o início – por exemplo, sempre importar bibliotecas nas primeiras células, usar nomes consistentes para conjuntos de treino e teste (X_train, X_test, etc.), organizar o notebook em seções bem-definidas – torna mais fácil converter esse experimento exploratório em um script de produção depois. Ou seja, estilo não é frescura, é uma forma de documentar e estruturar o pensamento.

Por fim, vale lembrar que convém conhecer e seguir os padrões da comunidade. No mundo Python, além da PEP 8, existe “o Zen of Python” (import this) com aforismos como “A legibilidade conta”. Já em R, há o tidyverse style guide, em Java a famosa convenção de chaves e indentação etc. Cada linguagem e empresa pode ter sutis diferenças nos padrões, mas todas compartilham o objetivo comum: código padronizado facilita comunicação.

Para o(a) cientista de dados, isso significa que seus experimentos não ficam como “caixa-preta” pessoal; pelo contrário, viram patrimônio compartilhado do time de desenvolvimento. Então, adote essas boas práticas e perceba como até mesmo seus scripts de análise ganharão vida nova – mais claros, confiáveis e prontos para crescer em escala. (Uma ótima referência é o artigo “How to Write Beautiful Python Code With PEP 8”, de Jasmine Finer, que explora detalhadamente cada

recomendação do guia, com exemplos práticos.) E lembre-se: se o lintern acusar um errinho aqui ou ali, não encare como crítica pessoal – é a oportunidade de deixar seu código mais elegante!

Princípios de Código Limpo: do DRY ao SOLID

Seguir um padrão de estilo é o primeiro passo, mas escrever um código realmente limpo vai além de formatação. Entra em cena a filosofia do Clean Code, fortemente propagada por Robert C. Martin (o “Uncle Bob”) e outros engenheiros experientes. Clean Code diz respeito a como estruturar e decompor o seu programa para que ele seja simples, coeso e livre de duplicações desnecessárias. Três máximas frequentemente lembradas – DRY, KISS e Single Responsibility – são verdadeiros mantras do desenvolvimento profissional e aplicá-las no contexto de ciência de dados pode turbinar suas pipelines e scripts analíticos.

DRY (Don’t Repeat Yourself): “Cada pedaço de informação deve ter uma representação única, não ambígua e definitiva no sistema”. Em termos práticos, evite código duplicado! Sempre que você se pega copiando e colando blocos de código, é sinal de que deveria refatorar aquilo para uma função ou módulo reutilizável. A motivação é matemática: se você tem N trechos duplicados e descobre um bug, terá que corrigir N lugares – e a chance de esquecer algum cresce com N .

Ao aplicar DRY, uma mudança necessária impacta apenas um ponto do código. No exemplo do Titanic que refatoramos no hands on, inicialmente havia duas funções avaliando modelos diferentes com lógica repetida; unificamos em uma só. Isso não apenas elimina redundâncias, mas reduz a probabilidade de inconsistências (imagine se você ajustasse a função de Random Forest e esquecesse de ajustar a de AdaBoost – resultados desequilibrados poderiam surgir).

O DRY também se aplica a datasets e configurações: em vez de definir um mesmo caminho de diretório ou parâmetro de treinamento em múltiplos scripts, defina-o uma vez (por exemplo, em um arquivo de configuração) e referencie isso em todos os locais. Assim, se houver alteração (digamos, mudar a fonte dos dados), você atualiza em um único lugar e todo o sistema reflete a mudança.

Ferramentas de limpeza de código podem ajudar a identificar duplicações – por exemplo, o flake8 com plugin de código copiado acusa partes muito similares. Um ponto importante: o DRY deve ser aplicado com bom senso; às vezes, tentar reutilizar código a todo custo pode levar a abstrações precoces ou confusas. Equilíbrio é tudo – falaremos mais sobre isso ao tratar de over-engineering.

KISS (Keep It Simple, Stupid): apesar do nome jocoso, o recado é sério – mantenha seu design o mais simples possível. Em outras palavras, não complique o que pode ser resolvido de forma direta. Em ciência da computação, muitas vezes a solução mais elegante para um problema é também a mais simples em termos de lógica. Por exemplo, se você precisa calcular uma média móvel, talvez não seja necessário implementar uma classe inteira com dezenas de métodos – uma função clara ou algumas linhas usando Pandas já bastam.

O KISS envolve não reinventar a roda e evitar excesso de funcionalidades desnecessárias. Matemáticos(as) gostam de dizer que as soluções mais bonitas são as mais simples; em código isso ecoa fortemente. Na prática, ao desenhar um módulo, pergunte: há uma forma mais direta de conseguir o mesmo resultado? Uma consequência de KISS é melhorar a performance: menos passos desnecessários significam menos operações para a CPU executar e menos alocação de memória.

Por exemplo, um código simples que utiliza operações vetorizadas do NumPy em vez de múltiplos loops em Python puro não só é mais limpo como também muito mais rápido, graças a operações nativas otimizadas. Manter simplicidade também facilita a verificação matemática do algoritmo – se seu código para calcular um desvio padrão está espalhado em 5 funções diferentes interligadas, é bem mais difícil assegurar que ele realmente corresponde à fórmula do que se estiver em um único bloco coeso.

O princípio KISS atua como um lembrete constante contra a tentação de “enfeitar” demais o código. Como bem coloca o ditado da engenharia: “menos é mais”. Isso não significa escrever menos linhas apenas, e sim manter a complexidade essencial mínima. Quando você sentir que está adicionando muitas camadas ou parâmetros “por via das dúvidas”, talvez esteja indo contra o KISS. Nesse caso, busque simplificar – muitas vezes, dividir um problema em partes menores já impõe simplicidade natural a cada parte. E lembre-se: um código simples

tende a ter menos bugs porque há menos cantos obscuros onde erros podem se esconder.

Single Responsibility Principle (SRP): este princípio, parte do famoso acrônimo SOLID, diz que cada módulo ou classe deve ter uma – e apenas uma – razão para mudar. Ou seja, concentre cada componente do seu software em uma responsabilidade bem definida. Se uma classe “faz tudo”, ela infringe SRP e se torna difícil de dar manutenção. Imagine um script de análise que baixa dados, faz limpeza, treina um modelo e também envia um e-mail com resultados – tudo junto. São tarefas bem distintas; segundo o SRP, deveríamos separar isso em componentes: um módulo de acesso a dados, outro de processamento, outro de modelagem e talvez uma função específica para notificação.

Assim, se houver mudança nos requisitos (ex.: trocar o algoritmo de modelagem ou a forma de notificar resultados), cada mudança ficará confinada ao módulo responsável, sem efeitos colaterais nos outros. Esse princípio é padronizado no desenvolvimento de software, mas vale muito para pipelines de ciência de dados. Ao projetar seu pipeline, pense em etapas bem definidas (extração, transformação, análise, visualização) e procure organizar o código em função disso.

Um benefício colateral é a facilidade de teste: componentes pequenos e focados são mais fáceis de testar individualmente (unit tests) – por exemplo, você pode testar separadamente a função que limpa os dados, garantindo que ela lida com valores nulos apropriadamente, antes mesmo de integrar com a etapa de modelagem. Além disso, o SRP favorece o reuso: se amanhã outro projeto precisar apenas da rotina de limpeza, você pode extraí-la e reutilizá-la sem arrastar dependências desnecessárias. Em resumo, coesão alta e acoplamento baixo: cada parte do código coesa em torno de seu propósito e interdependências mínimas entre partes distintas.

Esses três princípios (DRY, KISS, SRP) formam um núcleo de “código limpo” que reduz drasticamente a complexidade accidental dos sistemas. Mas há outros acrônimos notáveis: por exemplo, YAGNI (You Aren’t Gonna Need It – “você não vai precisar disso”) previne a tendência de implementar uma funcionalidade que talvez será útil no futuro, mas que no presente só adiciona complexidade sem valor. Em ciência de dados, é comum pensar “vou já preparar o código para suportar 10 tipos

de modelo, caso precisemos” – só que muitas vezes esse “caso” nem ocorre e você gastou tempo e encheu o código de configurações desnecessárias.

O YAGNI nos lembra de focar no requisito atual, deixando extensões para quando (e se) realmente forem necessárias. Isso anda de mãos dadas com KISS: evite sobrecarga de opções antecipadamente. Claro, não é para impedir evoluções – é apenas para não se antecipar demais a problemas hipotéticos. Adotar uma mentalidade YAGNI ajuda a evitar o temido over-engineering, ou superengenharia, que é quando o código fica excessivamente complexo porque o(a) desenvolvedor(a) tentou abarcar cenários demais ou usar abstrações sofisticadas sem necessidade real. Lembre-se: complexidade desnecessária é inimiga da manutenção.

Falando em complexidade, uma abordagem sistemática para mantê-la sob controle é praticar refatoração contínua. Refatorar significa melhorar a estrutura interna do código sem mudar seu comportamento externo. Na história contada por Ben Feifke (2023) sobre “Joe e Jane”, vimos que Jane periodicamente parava de adicionar novas funcionalidades para refatorar o código, eliminando duplicações e melhorando o design, enquanto Joe apenas jogava feature atrás de feature. O resultado: o progresso de Joe emperrou em bugs e enroscos, enquanto Jane, com uma base de código limpa, avançou cada vez mais rápido.

A moral é clara: “a única maneira de ir rápido é indo bem” (frase atribuída a Martin Fowler, citada no artigo). Para cientistas de dados, isso significa que mesmo em protótipos exploratórios vale a pena refatorar. Se você percebeu que escreveu o mesmo pedaço de lógica várias vezes durante uma análise, considere parar e criar uma função. Se um bloco de código ficou enorme realizando múltiplas tarefas (por exemplo, carregando dados, depois filtrando, depois plotando gráficos), avalie quebrá-lo em funções menores. Essa disciplina pode parecer adicionar trabalho no curto prazo, mas evita que o código se torne uma “bola de neve” ininteligível depois.

Além do mais, em ambientes de código compartilhado, seus colegas (ou você mesmo, após algumas semanas) vão agradecer por encontrar um código organizado e devidamente comentado, em vez de um emaranhado difícil de seguir. Clean Code envolve também prestar atenção a “bad smells” (maus cheiros do código) – indicadores sutis de que algo não está bem projetado. Exemplos: funções longas demais, muitas condicionais aninhadas, uso abundante de flags para alterar comportamento de função, variáveis com nomes genéricos (temp, data, value...) etc.

Quando sentir esses cheiros, aplique técnicas de refatoração conhecidas: extrair função (quando um trecho interno pode virar uma nova função), renomear para clareza, mover função para outro módulo mais coerente, reduzir níveis de aninhamento (por exemplo, usando retornos antecipados em vez de if/else encadeados). Cada pequena melhoria deixa o código mais limpo. E um código limpo é terreno fértil para eficiência: facilita otimizações pontuais (porque você entende melhor o fluxo, podendo identificar gargalos) e permite paralelização ou distribuição mais segura (por exemplo, quebrar um processamento em tarefas paralelas no Spark é viável apenas se as etapas estiverem bem definidas e isoladas).

Resumindo esta parte: escrever código limpo é quase uma engenharia de controle – você controla a entropia do sistema para que ele não se desorganize à medida que cresce. Em projetos de ciência de dados, especialmente quando evoluem de protótipo para produto, isso é crítico. Adotar convenções de estilo e princípios de design desde o início confere robustez e escalabilidade ao seu trabalho. E lembre-se: Clean Code não é algo que se faz uma vez e pronto – é um processo contínuo e uma mentalidade. Então, revise seu próprio código com frequência, procure feedback dos colegas e não hesite em refatorar quando necessário. Como conselho final desta seção, deixe anotado no seu checklist mental: “Não se repita, mantenha simples, separe responsabilidades” – e você evitará a maioria dos males que afligem códigos por aí.

Padrões de Projeto: reutilizando soluções consagradas

Até aqui falamos de princípios gerais de boa codificação. Agora, vamos subir um nível e discutir Design Patterns (Padrões de Projeto). São soluções arquiteturais para problemas recorrentes, catalogadas pela literatura de engenharia de software – notadamente pelo famoso “Gang of Four” (Gamma, Helm, Johnson, Vlissides, 1994) que documentou 23 padrões clássicos. A ideia por trás de padrões de projeto é ter modelos reutilizáveis: em vez de reinventar a estrutura de solução para cada problema, você identifica que certo desafio se encaixa em um padrão já conhecido e aplica aquele esqueleto de solução, adaptando ao contexto.

Isso ajuda a construir sistemas mais modulares, flexíveis e extensíveis. Entretanto, é importante dosar: aprendizado de padrões inclui também saber quando

NÃO usar – nosso objetivo não é sair aplicando padrão indiscriminadamente (evitando o over-engineering mencionado). Vamos focar em três padrões que são úteis de conhecer, inclusive em projetos de ciência de dados: Strategy, Observer e Factory. Para cada um, veremos o propósito, um exemplo simples (análogo ou em código) e como ele poderia aparecer no contexto de data science.

Strategy (Estratégia): é um padrão comportamental que encapsula algoritmos intercambiáveis. Em vez de ter um código monolítico cobrindo todos os casos, você define uma família de algoritmos (estratégias), cada um implementado em uma classe separada, e uma interface comum entre eles. O cliente (código que usa o algoritmo) mantém uma referência genérica à interface de estratégia, podendo em tempo de execução decidir qual estratégia concreta usar. Em termos simples, o Strategy permite selecionar o algoritmo em tempo de execução sem alterar o código cliente.

Por exemplo, no caso de validação de dados citado na Wikipedia: uma classe de Validador pode escolher entre diferentes algoritmos de validação dependendo do tipo de dado – esses algoritmos (e.g. validação numérica, validação de texto) são implementados separadamente e o Validador escolhe um deles conforme necessário. Como isso se relaciona a ciência de dados? Pense em um cenário de treinamento de modelos: você escreve uma função genérica de treinamento que aceita uma “estratégia de otimização” – podendo ser Gradiente Descendente, Adam, LBFGS etc. Cada estratégia de otimização segue a mesma interface (por exemplo, método `atualiza_pesos()`), mas você pode trocar qual está sendo usada sem mudar o laço de treino principal.

Na prática, muitas bibliotecas já fornecem isso (otimizadores intercambiáveis em deep learning frameworks, por exemplo), mas conhecer o padrão te ajuda a reconhecer e criar extensões. No hands on, nosso design genérico de `avaliar_modelo` é uma espécie de aplicação do conceito Strategy: passamos a classe do modelo como parâmetro – em outra abordagem poderíamos ter criado uma interface de Modelo com um método `train_and_score` e implementações específicas para RandomForest, AdaBoost, etc., depois passar instâncias dessa interface para uma função que espera o tipo `ModeloEstratégia`.

Esse design isolaria completamente a lógica de treinamento do algoritmo específico, deixando o código aberto a extensões (novos modelos) sem modificar a

função genérica – um exemplo do princípio aberto/fechado (OCP) em ação. Em resumo, use Strategy quando tiver múltiplas abordagens para um mesmo problema e quiser poder trocar facilmente entre elas. Só cuidado para não confundir com simples ifs: o benefício real do padrão aparece se as estratégias forem complexas ou numerosas, justificando organizá-las separadamente. Em um pipeline de ciência de dados, Strategy pode surgir ao definir diferentes estratégias de limpeza de dados aplicáveis conforme a fonte (planilha, banco SQL, API) – cada qual implementada em uma classe distinta, mas chamada de forma uniforme. Assim, seu código flui de maneira genérica, delegando as diferenças para as estratégias.

Observer (Observador): este padrão define uma relação do tipo um-para-muitos entre objetos, onde mudanças em um objeto principal (sujeito observado) disparam notificações para outros objetos dependentes (observadores). É muito utilizado em sistemas de eventos, interfaces gráficas e implementação de pub-sub (publicação/assinatura). A ideia central: o objeto Subject mantém uma lista de Observers e oferece métodos para registrá-los ou removê-los; quando o estado do Subject muda, ele notifica todos os observers, geralmente chamando um método padronizado (por exemplo, `atualiza()` ou `update()`) em cada um.

Um exemplo clássico: no padrão de arquitetura MVC, o Model é observado por múltiplas Views; quando os dados do Model mudam, todas as Views são informadas para se atualizar. O padrão Observer é útil quando diferentes partes do sistema precisam reagir a algo, mas queremos evitar polling constante ou dependências rígidas. Em ciência de dados, imaginemos um experimento rodando várias iterações de treinamento: poderíamos ter um Observer que registra métricas a cada época de treino.

O modelo (Subject) notifica “terminei a época 5, acurácia 0.8” e observadores diversos poderiam reagir – um para logar isso em um arquivo, outro para exibir em tempo real em um dashboard, outro para talvez ajustar automaticamente um parâmetro. Todos ficam sincronizados com o estado do modelo sem o modelo precisar saber detalhes de cada ação (ele só dispara o evento). Outro exemplo: suponha um objeto central que gerencia parâmetros globais de um experimento; vários módulos (pré-processamento, modelagem, visualização) observam esse gerenciador. Se um parâmetro for alterado (por exemplo, porcentagem de dados de

validação), todos os módulos observadores recebem a atualização e podem reagir (reparticionando dados, recalculando algo etc.).

Isso é preferível a ter cada módulo consultando constantemente o estado, ou a duplicar o estado em vários lugares. Em termos de implementação Python, o Observer pode ser feito de forma simples com callbacks ou utilizando bibliotecas (como o Observable do RxPY). Um cuidado: garantir que a notificação seja eficiente – se você tiver milhares de observers, a atualização pode ser custosa; mas na maioria dos casos em ciência de dados, o número de partes interessadas em um evento é pequeno. Resumindo, o Observer é indicado quando há propagação de mudanças: algo relevante acontece em um canto do sistema e várias outras partes precisam saber. Em vez de acoplar todos diretamente, usa-se esse mecanismo flexível de inscrição/notificação.

Factory (Fábrica): é um padrão criacional que fornece uma interface para criar objetos em vez de instanciá-los diretamente nas classes que precisam deles. Em essência, uma Factory abstrai o processo de construção de objetos complexos ou com lógica condicional, delegando a decisão de qual classe concreta instanciar. O exemplo típico é o Factory Method, em que uma classe mãe declara um método fábrica abstrato e subclasses específicas o implementam para retornar instâncias de diferentes classes.

Mas podemos pensar de forma mais simples: uma função fábrica `cria_modelo(tipo)` que retorna uma instância de um modelo de regressão ou classificação conforme um parâmetro `tipo` – internamente essa função decide `if tipo == 'arvore': return RandomForest() elif tipo == 'rede': return MLP() ...`. O código chamador não precisa saber desses detalhes, apenas usa a interface comum de modelo. A vantagem é centralizar a lógica de criação: se amanhã mudarmos a implementação de `RandomForest` para usar parâmetros específicos, fazemos isso na fábrica, mantendo o restante do código inalterado.

Em projetos de ciência de dados, um uso de Factory seria, por exemplo, na leitura de dados: imagine uma função `carrega_dados(fonte)` que pode receber `fonte="csv"` ou `"sql"` ou `"api"`. Em vez de espalhar condicionais pelo código cada vez que precisamos carregar dados de formatos distintos, uma Factory lida com isso – retornando um `DataFrame` independentemente da fonte, após usar internamente o

leitor adequado (Pandas, connector de banco, requisição HTTP etc.). Assim, o código de análise subsequente pode tratar todos os dados de forma uniforme.

Esse padrão promove o princípio aberto/fechado novamente: adicionar um novo tipo de objeto (ou nova fonte de dado) requer apenas criar uma nova classe concreta e ajustar a fábrica, sem modificar a lógica que usa os objetos. Outro exemplo: ao implementar uma aplicação web de visualização de resultados de modelos, você pode ter uma Factory que produz o gráfico apropriado conforme o tipo de variável (se categórica, cria um gráfico de barras; se contínua, um histograma, etc.).

O componente que solicita o gráfico não precisa ter um grande switch case para cada tipo – ele delega isso à fábrica e recebe um objeto gráfico padronizado (quem sabe um JSON ou figura pronta). Vale mencionar que o Python, por ser dinâmico, às vezes permite soluções mais diretas usando dicionários de classes ou funções em vez de classes Factory formais. Ainda assim, o conceito é válido: separar o ato de decidir/produzir o objeto da lógica de negócio que o utiliza. Isso aumenta a limpeza do código, pois cada parte faz uma coisa: a fábrica cria, o consumidor usa.

Conhecer esses padrões e outros (Singleton, Decorator, Adapter, etc.) fornece ao desenvolvedor um vocabulário para arquitetar soluções robustas. No entanto, um alerta importante: evite cair na armadilha de patternitis, que é quando desenvolvedores tentam encaixar um padrão de projeto a todo custo mesmo quando uma solução simples resolveria. Lembre-se do KISS e YAGNI: use padrões quando eles claramente beneficiam a qualidade e flexibilidade do código, não para “mostrar erudição”.

Por exemplo, não precisa implementar um Observer complexo se um simples callback ou sinal resolverá o problema de maneira clara. Muitas vezes, aprendemos sobre padrões lendo livros ou tutoriais, mas na prática a habilidade mais valiosa é identificar a oportunidade certa de aplicá-los. Uma dica: se você nota que está escrevendo muito código cerimonial ou repetitivo para lidar com variações de comportamento, ou gerenciando manualmente listas de ouvintes interessados em eventos, pode ser a hora de adotar um padrão (Strategy no primeiro caso, Observer no segundo, por exemplo). Outra dica: ao revisar código de terceiros ou bibliotecas

populares, tente reconhecer padrões de projeto ali – isso exercita seu olhar e inspira soluções.

Afinal, padrões de projeto não são exclusivos de Java ou C++; eles se aplicam em Python, R, Julia, ou qualquer linguagem (embora às vezes implementados de forma idiomática diferente). Em suma, os padrões de projeto são atalhos cognitivos: em vez de explicar “eu criei aqui um esquema onde escolho dinamicamente o algoritmo”, você pode dizer “usei o padrão Strategy” – e outros desenvolvedores já entenderão a essência. Nesta aula focamos em alguns padrões básicos, mas recomendamos a leitura do catálogo completo (como no livro “Design Patterns” do GoF ou em recursos online) e a prática em pequenos projetos pessoais para fixá-los.

Para fechar essa seção teórica, podemos pensar em como todos esses conceitos convergem no dia a dia de um(a) cientista de dados que escreve código de qualidade: ao iniciar um novo projeto ou componente, ele(a) define um estilo consistente (seguindo guias), aplica princípios de design para estruturar as partes (separa bem as responsabilidades, evita duplicações) e sabe que existem padrões consagrados caso o problema peça por uma solução mais elaborada de arquitetura. O resultado disso é código com menos bugs, mais eficiência (tanto de desenvolvimento quanto de execução) e uma base sólida para colaboração.

Em termos quantitativos, estudos indicam que equipes que adotam práticas de clean code e design adequado reduzem significativamente o tempo gasto com depuração e retrabalho, liberando mais tempo para focar em análise de dados e construção de modelos de fato. Lembre-se: código é também um ativo do projeto – tratando-o com o mesmo rigor que tratamos os dados, conseguimos escalar soluções de ciência de dados de um script isolado para sistemas inteiros em produção.

Você está, portanto, construindo não apenas um modelo ou relatório, mas um produto de software em essência. Padrões de código e estilo são as ferramentas que vão te ajudar nessa jornada, possibilitando que suas ideias brilhantes em data science se transformem em aplicações sustentáveis e de impacto no mundo real.

MERCADO, CASES E TENDÊNCIAS

IA Generativa e Automação de Código: ferramentas como ChatGPT e Claude estão sendo usadas para gerar insights e otimizar código, acelerando ciclos de desenvolvimento em projetos de dados ([MIT Sloan Review](#)).

Real-Time Data Engineering e Zero-ETL: arquiteturas orientadas a eventos e pipelines em tempo real são destaque, com ferramentas como Azure Stream Analytics, Databricks Structured Streaming e Microsoft Fabric Real-Time Analytics ([DataEX](#)).

DataOps e Governança Avançada: a aplicação de princípios DevOps à gestão de dados cresce, garantindo qualidade e rastreabilidade em ambientes complexos ([Coffee and Tips](#)).

Target (Varejo): uso de algoritmos preditivos para personalização de ofertas, aumentando vendas e fidelização ([Intercompany](#)).

Hospital Mount Sinai (Saúde): projeto “Deep Patient” aplicando machine learning para prever doenças com base em prontuários, melhorando diagnósticos.

Fracasso emblemático – Knight Capital (2012): perda de US\$ 440 milhões em 45 minutos por falhas de código legado e ausência de testes. Um alerta sobre a importância de boas práticas e governança.

“Best Practices for Scientific Computing” (Wilson et al., PLOS Biology): reforça princípios como DRY e modularidade para garantir reprodutibilidade científica.

Artigos sobre padrões de codificação: Alboukadel Kassambara (2024) discute estratégias para código limpo e sustentável ([Datanovia](#)).

Como desenvolver boas práticas de programação? – Conversa com Fabio Akita sobre legibilidade, refatoração e design patterns ([YouTube – Alura](#)).

Padrões de Projeto na Prática – Análise real de repositório com Strategy, Factory e SOLID ([Dialogando TI](#)).

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos como a combinação de boas práticas de estilo e padrões de projeto pode elevar o patamar do seu código em projetos de ciência de dados. Começamos discutindo convenções de código (em especial a PEP 8 no Python) e entendemos que escrever código legível e padronizado não é perfumaria, mas sim fundamental para colaboração e manutenção a longo prazo. Em seguida, mergulhamos nos princípios de Clean Code – aprendemos a evitar repetições desnecessárias com DRY, a buscar sempre a solução mais simples com KISS e a projetar funções/módulos focados em responsabilidades específicas (Single Responsibility).

Vimos que esses princípios reduzem bugs e tornam o desenvolvimento mais ágil conforme os projetos crescem. Abordamos também alguns Padrões de Projeto (Strategy, Observer, Factory), entendendo suas motivações e quando aplicá-los para resolver problemas recorrentes de forma elegante e extensível. Tudo isso foi ilustrado com exemplos práticos – inclusive um hands on onde refatoramos código duplicado em Python para torná-lo mais limpo e genérico.

Por fim, analisamos casos reais do mercado, reforçando a importância desses tópicos: desde um erro multimilionário causado por código mal gerenciado, até referências positivas de empresas e publicações que obtiveram sucesso com rigor e boas práticas. Em resumo, você aprendeu que escrever um código bem estruturado, claro e aderente a padrões não é apenas uma questão estética – é um diferencial crucial no trabalho em ciência de dados moderna.

Agora é hora de aplicar esses conceitos nos seus projetos: revise seus códigos antigos com esse olhar crítico, pratique refatorações e continue estudando. Assista às videoaulas quantas vezes precisar e não hesite em revisitar este material de apoio ao encarar desafios de codificação no futuro. Seu eu desenvolvedor – e todos os(as) colegas que lerão seu código – com certeza vão agradecer!

REFERÊNCIAS

FEIFKE, B. **Refactoring For Data Scientists: A Beginner's Guide**. 2023. Disponível em: <https://benfeifke.com/posts/refactoring-for-data-scientists-a-beginners-guide/>. Acesso em: 15 dez. 2025.

FINER, J. **How to Write Beautiful Python Code With PEP 8**. 2025. Disponível em: <https://realpython.com/python-pep8/>. Acesso em: 15 dez. 2025.

FREECODECAMP. **Clean Code explicado**: um guia prático para iniciantes. 2024. Disponível em: <https://www.freecodecamp.org/portuguese/news/clean-code-explicado-um-guia-pratico-para-iniciantes/>. Acesso em: 15 dez. 2025.

MARTIN, R. C. **Clean Code: A Handbook of Agile Software Craftsmanship**. Upper Saddle River: Prentice Hall, 2008. ("Código Limpo", tradução em português por Alta Books, 2009.)

TORRES NETO, F. S. **Design Patterns: Os Padrões de Projeto mais Usados**. 2025. Disponível em: <https://manual-si-ufc.blogspot.com/2025/03/design-patterns-os-padroes-de-projeto.html>. Acesso em: 15 dez. 2025.

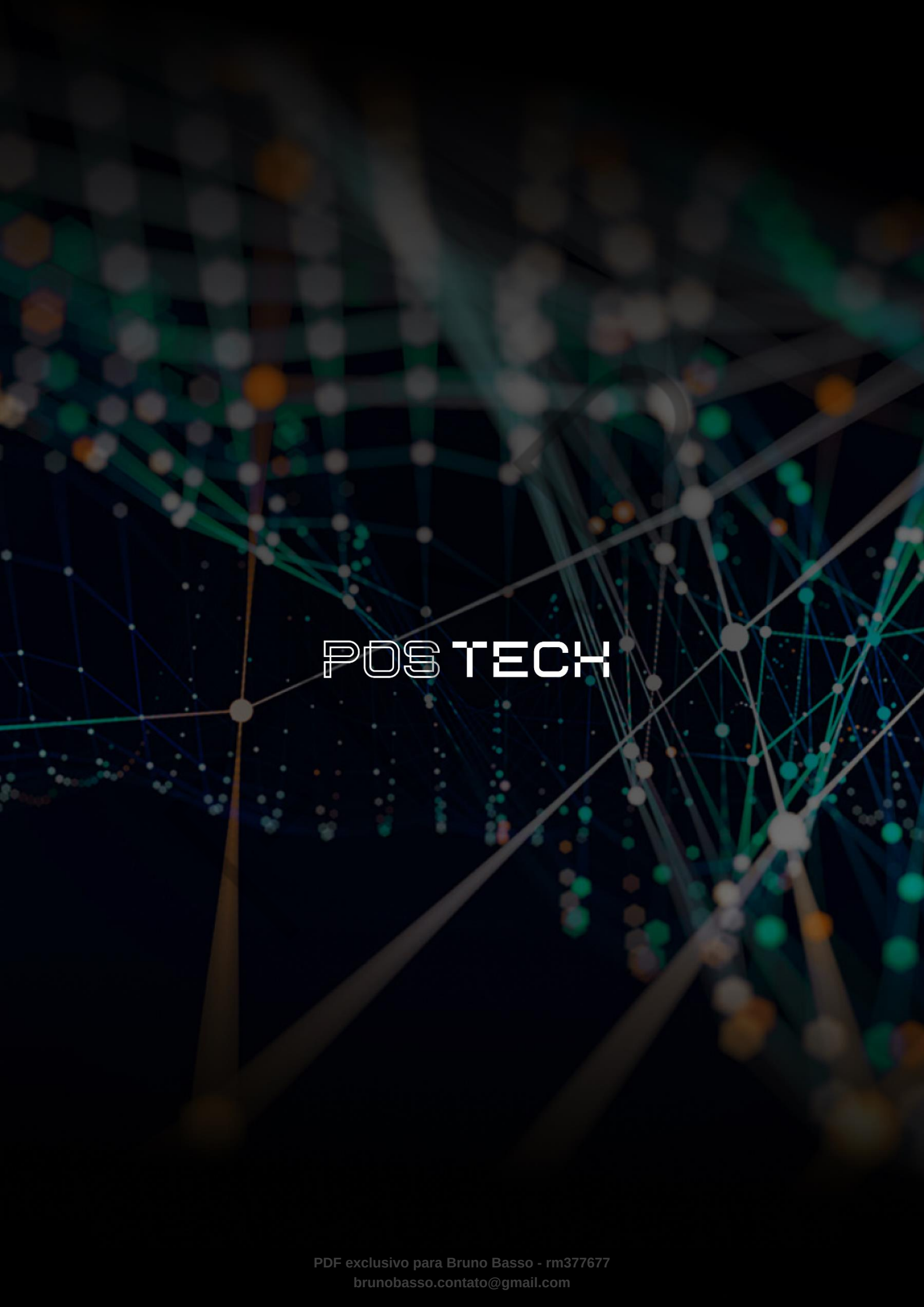
WATI. **O caso**: Knight Capital Group. 2025. Disponível em: <https://www.wati.com.br/post/o-caso-knight-capital-group>. Acesso em: 15 dez. 2025.

WILSON, G. et al. Best Practices for Scientific Computing. **PLoS Biology**, v. 12, n. 1, e1001745, 2014. Disponível em: <https://doi.org/10.1371/journal.pbio.1001745>. Acesso em: 15 dez. 2025.

PALAVRAS-CHAVE

Código Limpo. Padrões de Projeto. Manutenibilidade de Software.

EMSE



POSTECH